Rout Dutta

Pg 1

Saathi

Date 24/11/2025.

Project 4. ML → what does this mean?

CBIR → content based image retrieval (1)

Segmentation using clustering. → what does this mean?

(1) CBIR → locate samples from a database that are akin to a query, relying on content embedded within images.

⇒ (*) Strategy: calculating similarity b/w "compact vectors" by encoding both query & database images as global descriptors

(*) CLIP, UNICOM, CNN R GEM Swin → 1 semantic representation per image. [512-D, 768-D, 1024-D].

(*) what is "content"? → "sub regions"

content ⇒ high level semantics extracted from the backbone model.

what it includes:-

1. which objects are present?
2. their visual patterns.
3. overall style.
4. color distributions.
5. Texture
6. category cues.
7. shape structure.
8. scene type.
9. Relationship b/w subregions.

→ compressed learned representation

"ONLY latent features"

(*) What do these vectors contain?

1) object identity
2) structural & id / color attributes
3) global composition.
4) High level semantics
5) implicit local details.

DO NOT contain "bounding boxes" / "per pixel patches".

Saathi

Date __ / __ / ____

Q) How is a "png" image converted into a "global descriptor" vector?

Full pipeline:

→ gives out a set of vectors $g \in R^d$.

sizes: (commonly) {512, 768, 1024}.

STEP [I] Feed image into a backbone [CNN (1.) or ViT (2.)].

(i.) CNN → ResNet 101 → produces feature maps.
(ii) ViT → Patch embeddings → transformer outputs,
(iii) CLIP → image encoder → final pooled embedding

STEP [II] Aggregate the cluster of vectors into 1 vector per image.

Once we get a set of vectors using 2 types:-

1. CNN → ResNet / R-GEM {feature map $\in R^{C \times H \times W}$}
2. ViT/CLIP/UNICOM {Vision transformers} → {patch embeddings → transformers → output tokens}

Step 2: Then aggregation happens.

CASE [I] CNN —step 1→ $R^{C \times H \times W}$,

Image: $(P_1 \times P_2 \times 3)$.
C → channel [2048 channels].
H → 7
W → 7 } → (1) image separated into 7×7 = 49 "regions" — "regions".

(2) Each position $\in R^C$ dimensional vector representa^n.

(3) 49 vectors for EACH 2048 size rep vectors.
$F_{:,h,w} \in R^C \longrightarrow \{v_1, v_2 \dots v_{HW}\}$.

→ Aggregation for [1] → average pool: $g_c = \frac{1}{HW} \sum_{h,w} F_{c,h,w}$.

[2] max pool: $g_c = \max_{h,w} \{F_{c,h,w}\}$.

Page No.

Date ___ / ___ / ___

~~(Q) What is a "QUERY"?~~

→ these are pre-trained models
∴ 1) No learning.
2) No finetuning.
3) No weight updates.

→ single vector representation of 1 img

CASE [2]: Vision Transformers.

1) Image is cut into 16×16 / 14×14 patches.
2) 16 × 16 = N ——→ "sub-images" for [EACH] img sample.
3) Each of these "N" patches are fed into a Vision transformer: $\{\hat{x}^{D}_{N}\} \in \mathbb{R}^{D \times N}$ Transformer x^{D}_{CLS} → CLS token

~~(i) CLS token (classification token) → $\{x^D, x^D_1, x^D_2, \ldots, x^D_N\}$~~

Each patch embedding $\{\hat{x}_D\} \in \mathbb{R}^D$. Total number → $N = 16 \times 16$.	~~for the total of images in the entire dataset.~~

(4) $x_{CLS} \in \mathbb{R}^D$ · prepend it into vector array i.e $\{x^D_{CLS}, x^D_1, x^D_2, \ldots, x^D_N\}$.

(5) $x^{(L)}_{CLS} = f(x_{CLS}, x_1, x_2, \ldots, x_N)$.
→ [summarizing vector] after 'L' layers of transformer

(6) CLIP, UNICOM ——→ x^D_{CLS} of size D for 1 image.

(7) Aggregation NOT NECESSARY. x_{CLS} is SMART.

Now for a Dataset of size Δ we have Δ representation vectors for each image

STEP [III]. we have descriptor set for each constituent image within leaf nodes.

NOTE: what the heck is a QUERY?
(A) It is an image that asks: "Is this tree / car / animal in an images of your dataset", "can you tell which one?", "which one do you think matches the closest description?".

Next step: Generate query Descriptor similarly using STEP [I] & STEP [II].

PIPELINE · PIPELINE

Date ___ / ___ / ___

~~Now are~~

~~STEP [IV]:~~

~~Use the global descriptor of QUERY (V_Q) & calculate similarity (S_i) with leaf nodes. Each leaf node of each image gives some similarity.~~

~~Similarity distance metric / cosine} Dataset size.~~

~~$V_Q \to \{V_1, V_2, V_3, \ldots V_i \ldots V_N\}$. N leaves — Dataset size.~~

~~$\Rightarrow \{S_1, S_2, S_3 \ldots S_i \ldots S_N\}$ scalars~~

~~Speed — Accuracy tradeoff.~~

STEP [IV]: OFFLINE Stage. (N → total dataset)

→ Now we have to understand that we start with OFFLINE Stage.

→ For CNN path ⟶ aggregated info rich vector.
For ViT path ⟶ CLS vector.

→ Now we have N such vectors.

→ Implement HKM → Heirarchical K-means. AND build search TREE.

→ ROOT node ⟶ K children. {μ_i} centroids.

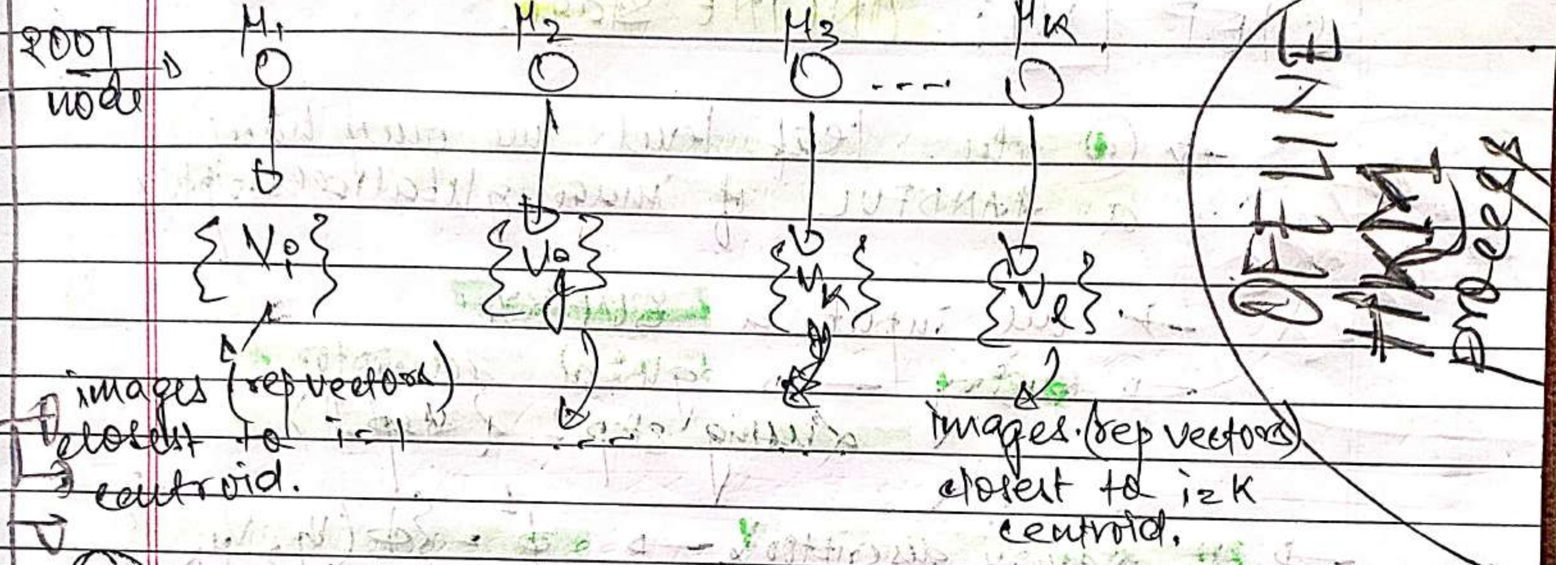
→ Each i^{th} node has images which are closest to the i^{th} centroid. ($\mu_i \Rightarrow$ (N)).

OFFLINE

Page No.

Hyperparams:- 1. K ⇒ K-means
↳ branching factor (AKA).
2. l ⇒ tree depth.

Date ___/___/___



(*) Run iterative K means and correctly bucket images. After recursive K-means, we finally get the correct buckets (final K-buckets). each bucket has $\{b_i\}$ number of images.

$$\sum_{i=1}^{m} b_i^{(1)} = N$$

(*) For $l \geq 2$, we do K-means again, again get K buckets for each child.

This recursion is dependent upon "l" leaf/tree depth.

$$\sum_{j=1}^{m} b_{i,j}^{(2)} = b_i^{(1)} \Rightarrow \sum_{k=1}^{m} b_{i,j,k}^{(3)} = b_{i,j}^{(2)}$$

$$\Downarrow$$

$$\sum_{a=1}^{m} b_{ijkl,a}^{(5)} = b_{ijkl} \Leftarrow \sum_{l=1}^{m} b_{ijk,l}^{(4)} = b_{ijk}^{(3)}$$

(*) Each of these buckets at any depth WILL only contain A VERY SMALL subset of images.

(*) Leaf nodes → store ACTUAL image vectors
INTERNAL nodes → "label" → centroid vectors @ each level.

Pg 6

Saathi

Date __/__/____

STEP V: ONLINE Stage.

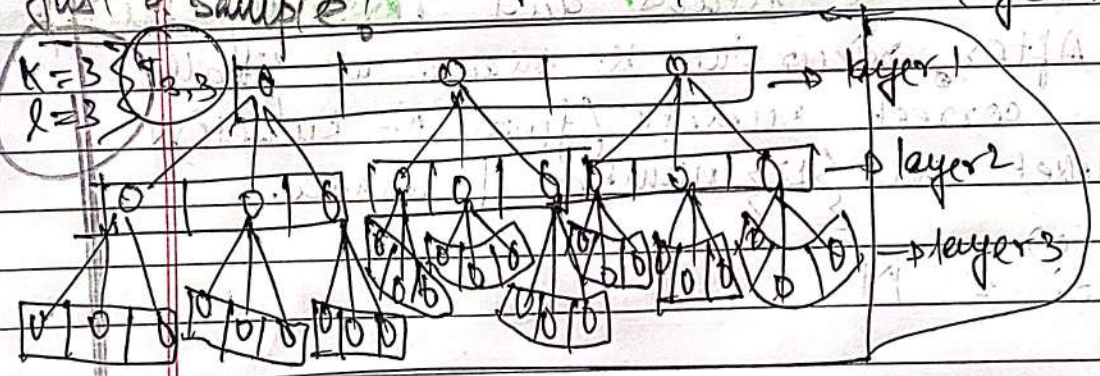
→ @ the leaf level we only have a HANDFUL of image global descriptors

→ we input a QUERY.

→ Query → global descriptor (using step I & step II).

→ V_Q (query descriptor) → $\phi^1 = \{\phi(V_Q, V_{M_K})\}$

just a sample!



choose argmin(ϕ^1).

choose argmin(ϕ^2) → move on to layer 2

→ move over to layer 3 → choose argmin(ϕ^3).

→ TIU Now the similarity metrics were b/w V_Q & $V_{M_K}^{(i)}$. Now that we have THE best Node. we choose THE candidate images in THAT node & calculate similarity vector.

→ suppose BEST node has β images, we calculate pairwise similarity → $\{\phi_{Q1}, \phi_{Q2}, \phi_{Q3} \cdots \phi_{Q\beta}\}$.

AND then SORT these images.

Page No.

Saathi

Date ___ / ___ / _____

→ sorted $\{s_{q_1}, s_{q_2}, s_{q_3} \cdots s_{q_B}\}$ ⇒ RANKING.

THAT'S IT. Now, argmax [sorted $\{s_{q_i}\}$] = ANS

answer